

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 09 · LECTURE NOTES

Integer Programming

NP-hard, and solved every day.

MODULE OUTCOMES

- Formulate with integer and binary variables.
- Explain branch and bound and what makes it work.
- Explain cutting planes and strengthening.
- Explain the integrality gap and formulation strength.
- Model the standard logical constructs.

1 Branch and bound

```
solve the LP RELAXATION (drop integrality)
-> a LOWER bound on the true minimum (Module 04, weak duality)
-> if the solution is already integral, DONE

otherwise pick a fractional variable  $x_j = 3.4$  and BRANCH
child A:  $x_j \leq 3$       child B:  $x_j \geq 4$ 
the children partition the region and exclude the fractional
point

maintain an INCUMBENT -- the best integral solution so far,
which is an UPPER bound

PRUNE a node when
  its LP bound exceeds the incumbent    -- nothing better inside
  its LP is infeasible                  -- nothing there at all
  its LP solution is integral            -- update the incumbent

The GAP between the best remaining bound and the incumbent is
reported CONTINUOUSLY. You can stop early with a guarantee:
"within 2% of optimal" is a complete, checkable claim.
```

Stopping early with a proven gap is what makes integer programming practical. You rarely need the exact optimum — a schedule 1% from optimal is operationally indistinguishable from the best one — **and you always need to know how far off you are.** That combination is why NP-hardness does not prevent daily industrial use (Module 01 §2), and it is the practical heart of this module.

THE BOUND QUALITY DECIDES EVERYTHING

A tight relaxation prunes aggressively and the search tree stays small. A loose one prunes almost nothing and the tree grows exponentially — the difference between seconds and never.

So formulation strength matters more than anything else. Two mathematically correct formulations of the same problem can differ by orders of magnitude in solve time, purely through the quality of their relaxations — which means modelling is not a preliminary to solving but the largest part of it (§3, Module 12).

Branching choice matters too. Which fractional variable to split on determines the tree's shape; **pseudo-cost branching** — estimating each variable's historical effect on the bound — is the standard heuristic and is substantially better than picking the most fractional variable.

And obtaining a good incumbent early helps enormously, because pruning requires something to prune *against* — with no incumbent, no node can be discarded. **Which is why solvers run primal heuristics before and throughout the search,** spending real time looking for merely good solutions in order to make the proof of optimality cheap.

2 Cutting planes

A CUT REMOVES FRACTIONAL POINTS AND NO INTEGER ONES

Add an inequality that is satisfied by every integer-feasible point but violated by the current fractional LP solution. The cut is *valid* (loses no real solutions) and *separating* (cuts off the current one).

The relaxation becomes tighter, the bound improves, and nothing feasible is lost. Re-solving the LP gives a better bound and possibly an integral solution.

Gomory cuts are derived mechanically from the simplex tableau and always exist, and **a pure cutting-plane method provably terminates** — but converges so slowly, and suffers such numerical difficulty, that it is unusable alone. This was known by 1960 and the method was abandoned for thirty years.

So modern solvers do branch and cut: generate cuts at the root and at nodes to tighten the local relaxation, then branch. **Problem-specific cuts — clique, cover, flow cover, mixed-integer rounding — are far stronger than general-purpose ones,** which is why a solver that recognises your problem's structure vastly outperforms one that does not, and why stating structure explicitly in the model pays.

$$\text{gap} = (z_{\text{IP}} - z_{\text{LP}}) / z_{\text{IP}}$$

The integrality gap measures how far the relaxation understates the true optimum, and it is the quantity that predicts solve difficulty. **The ideal formulation is the convex hull of the integer feasible points** — its LP relaxation has gap zero, every vertex is integral, and no branching is required at all (which is exactly the totally unimodular case of Module 03 §3, where the hull is described by the original constraints). **For a general problem the hull has exponentially many facets,** so it cannot be written down — **and every valid inequality you add moves the relaxation toward it. That is what cuts are for,**

geometrically, and it unifies cuts, formulation strength, and the gap under one picture.

3 Modelling with integers

WITH BINARY y :

```
either-or      x <= u*y,  x' <= u*(1-y)
if-then       y1 <= y2      ("y1 implies y2")
at most k of n  sum y_i <= k
fixed charge   cost = f*y + c*x,  x <= M*y
               -- pay the fixed cost f only if x > 0
piecewise linear SOS2 constraints, or a binary per segment
```

THE BIG-M TRAP

" $x \leq M*y$ " forces $x = 0$ when $y = 0$, for ANY sufficiently large M . It is CORRECT for any large M .

BUT a loose M makes the LP RELAXATION WEAK. With $M = 10^6$ and x bounded by 10 in reality, setting $y = 0.00001$ satisfies the constraint, the relaxation learns nothing, the bound is useless, and the tree explodes.

USE THE SMALLEST VALID M , derived from the problem's real bounds. This single discipline routinely changes solve times by orders of magnitude.

Big-M is the clearest available case of formulation strength mattering: the constraint is logically correct for any large M and *useful* only for a tight one, so correctness and performance come apart completely. **It is the most common modelling error in practice**, partly because the symptom — a solver that runs forever on a correct model — does not point at the cause.

4 In practice

- **Tighten every big-M.** Derive each from the problem's real bounds rather than picking a safe large number. **The highest-value single action available** (§3).
- **Break symmetry.** n identical machines, bins, or shifts produce $n!$ equivalent solutions, and the search tree will visit all of them unless prevented — add ordering constraints (machine 1's load $\geq$ machine 2's, or lexicographic ordering on assignment vectors) so each distinct solution is explored once. **Symmetry is the quiet killer of integer models**, and it is invisible until you look for it.
- **Supply a good incumbent** from a heuristic or from a previous run, so that pruning can begin at the first node rather than after the solver has found something itself.
- **Set a gap tolerance and a time limit.** **Proving the last 0.1% of optimality frequently costs more than finding the first 99.9%**, because the remaining nodes are precisely the ones the bound cannot distinguish. A 1% tolerance is usually generous and usually free.
- **And reformulate before tuning solver parameters.** **The formulation is worth more than every parameter setting combined**, and parameter tuning is where people reach first because it feels like action.

WHAT HAS ACTUALLY MADE SOLVERS FAST

Between roughly 1990 and 2020, integer programming solvers improved by a factor of about a million on standard benchmark sets. This has been measured repeatedly by running old solver versions on modern hardware.

Hardware contributed roughly a factor of a thousand. The algorithms contributed the other thousand. The two are comparable, which is an unusual and striking result — in most fields hardware dominates.

And the algorithmic gains were mostly cutting planes, presolve, and primal heuristics — not a better branching rule, not a faster LP solve, and not a fundamentally new search strategy. **The gains came from making each node's bound tighter and from finding good solutions sooner.**

So the practical lesson matches the theoretical one: strengthening the formulation is where the gains are, whether the solver's presolve does it automatically or you do it by hand. **And it is a reason to keep solvers updated,** which organisations frequently do not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 15.093 — integer programming lectures (free, OCW)	Branch and bound, cutting planes, and formulation strength — the core of §1 and §2.
Wolsey — Integer Programming	The standard reference, strong on formulation and on the convex hull view of §2.
Klotz & Newman — Practical Guidelines for Solving Difficult Mixed Integer Linear Programs (free)	The §4 checklist, from solver developers. The big-M and symmetry advice is here with worked examples.
Bixby — Solving Real-World Linear Programs, and the MIP benchmark histories (free)	The §4 callout's measurements, and current solver benchmarks.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement branch and bound** for small integer programs and report node counts.
- 2 **Solve the LP relaxation and measure the integrality gap** on several instances.

- 3 **Formulate one problem two ways** — a weak and a strong formulation — and compare node counts and solve times.
- 4 Implement a simple cover cut and measure its effect on the bound.
- 5 **Set up a big-M model with a loose M**, then tighten it, and report the solve times.
- 6 **Construct a symmetric instance** with identical items and measure the tree size with and without symmetry-breaking constraints.
- 7 Supply an incumbent from a greedy heuristic and measure the effect.
- 8 **Plot the gap against time** for a hard instance and identify where the last 1% begins.
- 9 Model three logical constructs from §3 and verify each.
- 10 **Compare an old solver version against a current one** on the same instances, if you can obtain both.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe branch and bound and name the three pruning conditions.
- 2 Why can you stop early with a guarantee, and why does that matter?
- 3 What decides the tree size?
- 4 What is a cutting plane, and what two properties must it have?
- 5 Define the integrality gap and relate cuts to the convex hull.
- 6 Give five logical constructs as linear constraints.
- 7 Explain the big-M trap.
- 8 Give five practical steps for a hard integer program.
- 9 What has made solvers fast, and in what proportion?